

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	S. Dharchini	Reg. No.:	192511260
Section / Batch:	2025-2nd year	Date:	04/09/2026

Code of Conduct

I S. Dharchini (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Dharchini

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

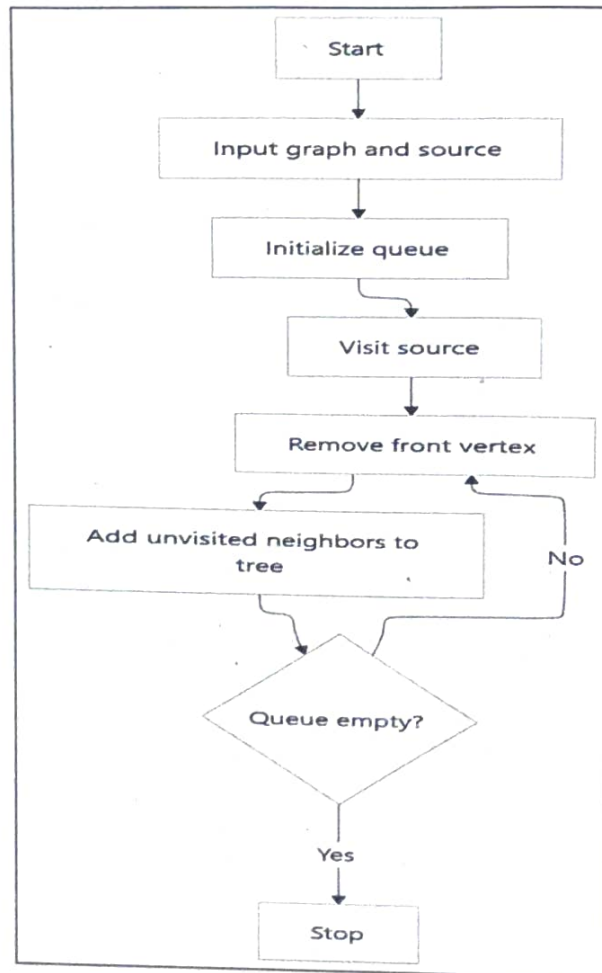
Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Shortest Path Algorithm	Dijkstra's Algorithm	2	20
Shortest Path Algorithm	Unweighted Graph	3	20
Graph Traversal	BFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A - Pseudocode Writing Task

1. Convert the flowchart for generating a BFS tree into code.

(20 Marks)

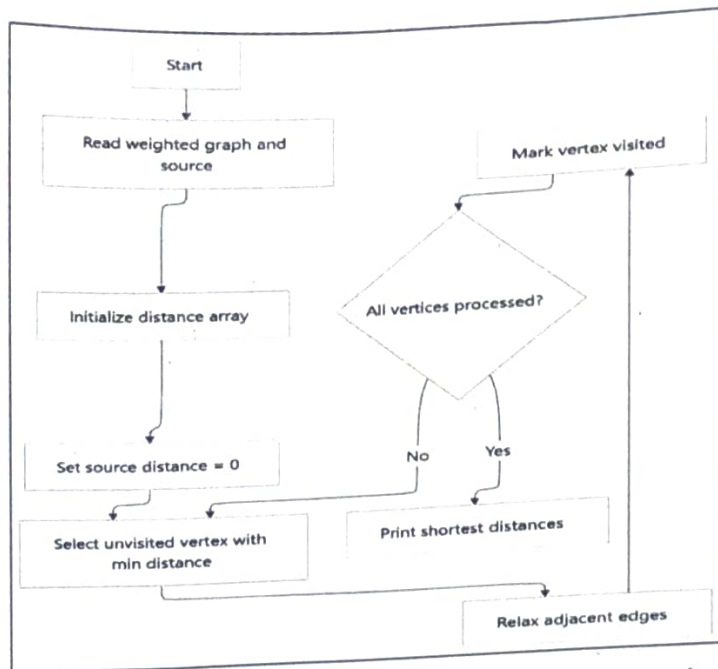
Flowchart Diagram:



2. Convert the flowchart for Dijkstra's shortest path algorithm into code.

(20 Marks)

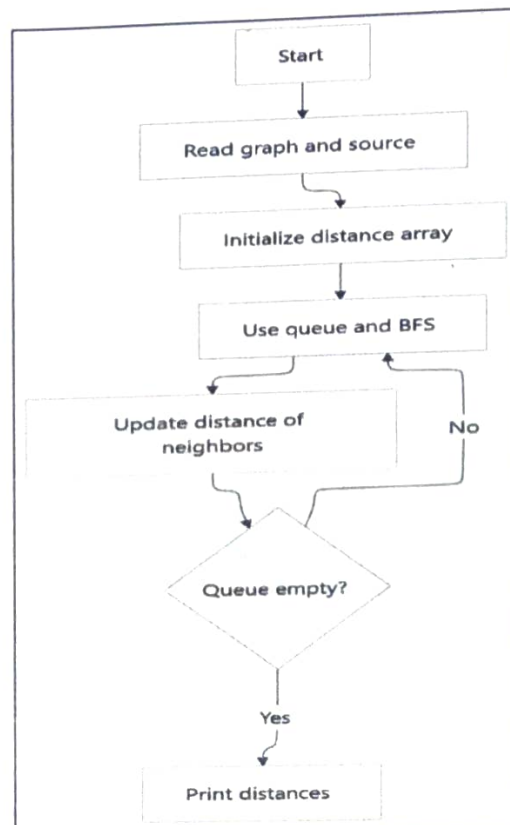
Flowchart Diagram:



3. Convert the flowchart for shortest path in an unweighted graph into code.

(20 Marks)

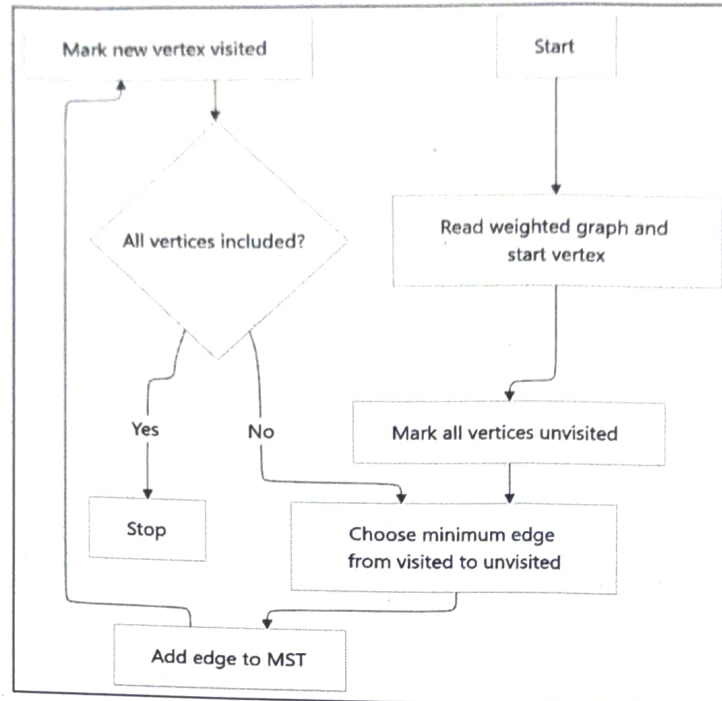
Flowchart Diagram:



4. Convert the flowchart for Prim's algorithm into code.

(20 Marks)

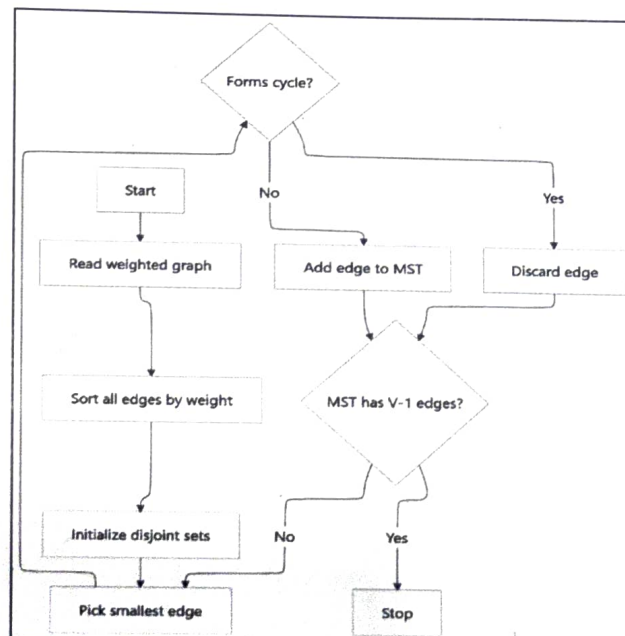
Flowchart Diagram:



5. Convert the flowchart for Kruskal's algorithm into code.

(20 Marks)

Flowchart Diagram:



1.

BFS Tree Generation

BEGIN

INPUT graph, source
 CREATE empty queue Q
 CREATE empty set visited
 CREATE empty tree T

ENQUEUE (Q, source)
 MARK source as visited

WHILE Q is not empty:

 CURRENT = DEQUEUE (Q)
 FOR each neighbor in graph [current]:

 IF neighbor not in visited:

 MARK neighbor as visited

 ADD edge (current, neighbor) to T

 ENQUEUE (Q, neighbor)

OUTPUT T

END

2.

DIJKSTRA'S SHORTEST PATH

BEGIN

READ weighted graph, source

INITIALIZE distance[v] = infinity for all v

SET distance[source] = 0

CREATE visited = empty set

WHILE all vertices not processed:

 U = select unvisited vertex with minimum distance [U]

 FOR each neighbor V of U:

 IF V not visited AND distance[U] + weight (U, V) <

 distance[V]:

 MARK V as visited

PRINT distance []

END

3.

SHORTEST PATH IN A UNWEIGHTED

BEGIN

READ graph, source

INITIALIZE $\text{distance}[v] = \infty$ for all v

SET $\text{distance}[\text{source}] = 0$

CREATE Queue Q, ENQUEUE (source)

WHILE Q is not empty:

 CURRENT = DEQUEUE (Q)

 FOR each neighbor of current:-

 IF $\text{distance}[\text{neighbor}] = \infty$:

$\text{distance}[\text{neighbor}] = \text{distance}[\text{current}] + 1$

 ENQUEUE (Q, neighbor)

PRINT $\text{distance}[]$

END

4.

PRIM'S ALGORITHM

BEGIN

READ weighted graph, start-vertex

MARK all vertices as unvisited

MARK start-vertex as visited

INITIALISE MST = empty set

WHILE not all vertices included:-

 edge (u,v) = CHOOSE minimum weight edge where u
is visited, v is unvisited

 ADD edge (u,v) to MST

 MARK v as visited

OUTPUT MST

END

KRUSKAL'S ALGORITHM

BEGIN

READ weighted graph (V vertices, E edges)
SORT all edges in non-decreasing order of weight
INITIALIZE disjoint sets, one per vertex
INITIALISE $MST = \text{empty set}$

WHILE MST does not have $(V-1)$ edges AND edges remain:-
edge $(u, v) = \text{pick next smallest edge}$

IF $FIND(u) \neq FIND(v)$:

DISCARD edge

ELSE :

ADD edge (u, v) to MST

UNION (u, v)

OUTPUT MST

END.